

12.Seguridad en PHP

Primera recomendación: **Disponer siempre de versiones actualizadas de Apache y PHP**

Aspectos a tener en cuenta:

1. FILTRADO DE DATOS: **Validar** y **Sanear** los datos que nos llegan

La inserción de datos “extraños” es una forma de “atacar” nuestra aplicación, hay que validar y controlar todas las entradas de datos que recibimos:

- Los datos deben tener el formato correcto: números, letras, email....
- Los datos no deben incluir caracteres “ilegales”: para evitar que nos inyecten código javascript ([Cross-site scripting - XSS](#)). Es una vulnerabilidad que afecta más al cliente que al servidor

index.php

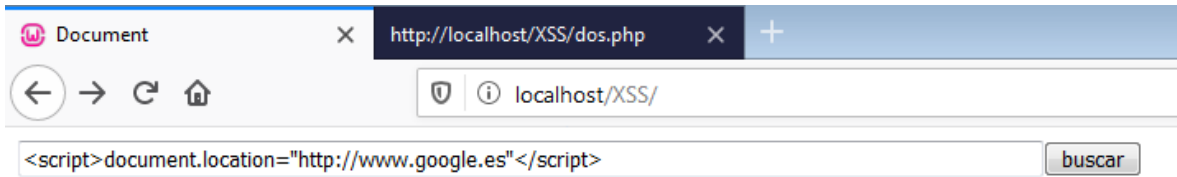
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Document</title>
</head>
<body>
  <form action="dos.php" method="post">
    <input type="text" name="texto"><input type="submit" name="buscar" value="buscar">
  </form>
</body>
</html>
```

dos.php

```
<?php
$patron=$_POST['texto'];

echo "Resultado de buscar $patron, no se encontró";
?>
```

¿Qué pasaría si introduzco en la caja texto <script>alert("Holaaaa")</script>?



Mediante los filtros en PHP aseguramos que los datos para nuestra aplicación sean los correctos.

<https://www.php.net/manual/es/filter.filters.php>

Son un conjunto de funciones que nos permiten sanear los datos y luego validar que sean correctos:

```
<?php
$email="pepe<script>alert('hola')</script>@gmail.com";
$email=filter_var($email,FILTER_SANITIZE_EMAIL);
echo "Tu email: $email<br>";
if ( !filter_var($email, FILTER_VALIDATE_EMAIL))
    echo "NO";
else
    echo "SI"
?>
```

2. ENCRIPtar CONTRASEÑAS : Veremos funciones hash
3. INYECCIÓN SQL: escapar todas las sentencias
4. ARCHIVOS DE CONFIGURACIÓN

Siempre con extensión php, no en la raíz del proyecto

5. CONFIGURACIÓN DE ENTORNOS DE DESARROLLO Y PRODUCCIÓN

Cuando una aplicación lanza errores por lo general muestra la ruta donde falló, por esto es importante que al desarrollar aplicaciones se cambie la configuración de archivos en el servidor. Cuándo estamos desarrollando

```
1 display_errors = On
2 display_startup_errors = On
3 error_reporting = -1
4 log_errors = On
```

En cambio cuando estamos en producción cambiamos

```
1 display_errors = Off
2 display_startup_errors = Off
3 error_reporting = E_ALL
4 log_errors = On
```

Con esto se seguirán registrando los errores a nivel del servidor, pero no serán visibles al usuario.

Estas configuraciones están en el archivo php.ini

6. VARIABLES GLOBALES

Cuando `register_globals` está activado en el fichero `php.ini`, PHP crea automáticamente variables globales a partir de los datos de los formularios y de las cookies. Esto puede dar lugar a problemas porque dichas variables pueden ser accedidas por cualquiera desde la URL.

Para resolver este problema existen tres soluciones:

- Deshabilitar `register_globals` en el fichero `php.ini` : esta directiva establece si se admite o no la creación automática de variables globales. Desde PHP 4.2.0 su valor por defecto es `off`
- Inicializar las variables: de esta manera aunque intenten acceder a tu página habiéndole dado un valor a la variable global, al iniciarse el código dicha variable tomará el que tú le des.

```
<?php
    // $usuario=false;
    if ( isUsuario() )
        $usuario=true;
?>
```

Si la llamada a mi página es `ejemplo.php?usuario=1`;

Se puede usar la directiva `error_reporting=E_ALL` en `php.ini` para que se muestre un aviso cuando se use una variable que no haya sido previamente inicializada.

- Establecer el orden de las variables : PHP crea automáticamente variables globales a partir del entorno (E), las cookies (C), la información del servidor (S) y los parámetros GET (G) y POST (P)

La directiva `variables_order` controla el orden de estas variables. El valor por defecto es "EGPCS". Permitir la creación de variables globales desde parámetros GET y POST y desde cookies es **potencialmente peligroso**. Un posible valor para `variables_order` que evita esto es "ES". En tal caso para acceder a los parámetros de los formularios y a las cookies se deben utilizar los arrays globales `$_GET`, `$_POST` y `$_COOKIES`

Si se modifican las directivas `register_globals` y/o `variables_order` es preciso revisar los scripts existentes para adaptarlos a las nuevas circunstancias.

7. NOMBRES DE FICHEROS

Las funciones de manejo de ficheros como `include()` o `require()` admiten nombres de ficheros remotos, lo que podría provocar la ejecución de código maligno cargado de otro servidor.

```
include ($libdir . "/conecta.php"); ,,  
Si un atacante pone $libdir= "http://atacante/", y coloca en la raíz del mismo un fichero  
de nombre conecta.php, su código sería ejecutado
```

Se puede desactivar la funcionalidad de acceso a ficheros remotos con la siguiente directiva en `php.ini`:

```
allow_url_include = off (así viene por defecto)
```

Para chequear nombres de ficheros se utilizan las funciones `realpath()` y `basename()`. La primera convierte direcciones relativas en absolutas y la segunda toma una ruta y devuelve la parte correspondiente al nombre del fichero.

```
$file = $_POST['username'];  
$file2 = basename (realpath($file));  
if ($file2 != $file) die ("$file no es un username válido");
```

Otra posibilidad frente a los nombres de ficheros incorrectos es la directiva de `php.ini` `open_basedir`

```
open_basedir = /alguna/ruta
```

PHP limitará las operaciones sobre ficheros al directorio especificado y sus subdirectorios:

```
include ("/alguna/ruta/lib.inc"); // permitido  
include ("/otra/ruta/lib.inc"); // da error
```

8. SUBIDA DE ARCHIVOS

La subida de ficheros permite a un usuario enviar cualquier fichero al servidor, esto es peligroso, se puede subir un código maligno y luego ejecutarlo.

Como recomendación general, debe evitarse utilizar el nombre enviado por el navegador, es conveniente generar un nombre único para el fichero subido.

Otro peligro es el tamaño de los ficheros. Aunque se limite el tamaño máximo en el código PHP, los ficheros se reciben automáticamente y luego comprobamos su tamaño. Es posible que un usuario intente provocar un ataque de denegación de servicio enviando varios ficheros de gran tamaño a la vez, llenando así el sistema de ficheros utilizado por PHP para almacenarlos. Para evitar esto se puede utilizar la directiva del php.ini

```
post_max_size
```

9. BIBLIOTECAS

Hay que almacenar fuera de la raíz de la web aquellos ficheros que contienen código php que usamos como biblioteca (por ejemplo nuestro archivo para conectar a una bbdd), para evitar que puedan ser accedidos por su URL.

Esto implica que hay que darle a las funciones include y require la ruta completa, lo que no es conveniente. Es mejor utilizar la directiva del php.ini ,include_path

```
include_path = ".:usr/local/php:usr/local/lib/myapp"
```

Determina la ruta en la que las funciones include, require, fopen buscarán el archivo que se les pasa.

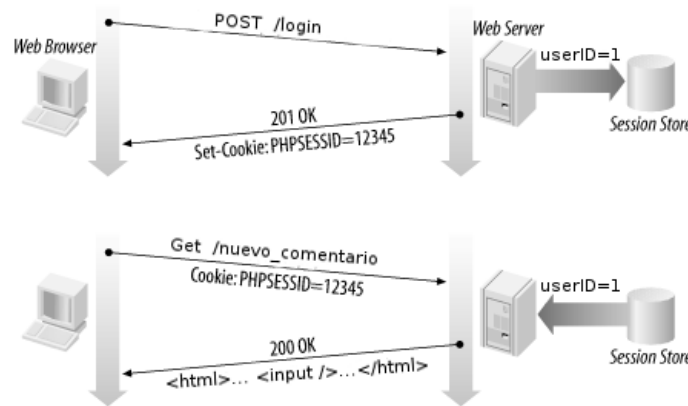
Se puede determinar desde código para cada aplicación con la función set_include_path()

10. SESIONES

Cada sesión viene identificada por un id aleatorio (32 caracteres hexadecimales), que se crea la inicializar dicha sesión con la función session_start(), y se almacena en una variable llamada PHPSESSID.

Este id es esencial porque cuando creamos una sesión, se crea un arraysuperglobal \$_SESSION que almacenará la información de la sesión. Este array se almacena en el servidor, en un archivo temporal cuyo nombre está formado por 'sess' + dicho ID de sesión (depende de la configuración de php.ini).

El servidor tiene que saber que las peticiones del cliente pertenecen a la sesión.



Hay dos posibilidades para comunicar el identificador de sesión:

1. Si el cliente tiene las cookies activadas, se crea una cookie con nombre PHPSESSID (se le puede cambiar el nombre en `php.ini`) cuyo valor es el identificador de 32 caracteres hexadecimales. Para crearla el servidor, tras haber creado la sesión (`sesión_start()`), envía la primera respuesta con la cookie en su cabecera:

```
HTTP/1.1 200 OK
Date: Sun, 12 Sep 2010 22:46:25 GMT
Server: Apache/2.2.15 (Win32) PHP/5.2.13
X-Powered-By: PHP/5.2.13
Set-Cookie: PHPSESSID=q75n4oq6turdv67uohqkb171v5; path=/
Expires: Thu, 5 Sep 2013 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0
Pragma: no-cache
Content-Length: 2031
Content-Type: text/html
<!DOCTYPEhtml PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<htmlxmlns="http://www.w3.org/1999/xhtml" lang="es" xml:lang="es">
<head>
<title>Página 1</title>
...
```

A partir de este momento, mientras dure la sesión, el navegador acompañará esta cookie con las solicitudes de páginas que se hagan a ese mismo servidor. Y este último comprobará que el identificador existe y es válido recuperando la sesión de entre sus ficheros donde se almacenan las sesiones.

2. Si el cliente (navegador) tiene las cookies desactivadas, PHP pasa el PHPSESSID en la URL junto con las llamadas a las páginas siguientes. Y para ello tenemos 2 opciones:

Añadir el PHPSESSID manualmente cada vez que creamos un enlace:

```
<a href="page.php?PHPSESSID=<?php echo htmlspecialchars(session_id());  
?>">haga click</a>
```

Dejar que PHP lo añada automáticamente. Si modificamos en el php.ini la variable *session.use_trans_sid* a True, PHP añadirá automáticamente el PHPSESSID a las URL de tu sitio web de manera transparente al desarrollador, siempre que la sesión no haya expirado y que las cookies del cliente estén desactivadas.

Esta opción viene desactivada por seguridad ya que estamos mostrando dicho id en la url. Enviar de un lado a otro el ID de Sesión es un problema porque puede ser interceptado y con ello se puede acceder a un servidor de forma ilegítima. Cuando un usuario se autentica en dicha aplicación el valor del ID de sesión que se creará no cambiará en toda la sesión (si no lo cambiamos nosotros). Un ataque de sesiones aquel en el que un usuario es capaz de robar la sesión de otro usuario y suplantar su identidad.

Qué hacemos para proteger nuestras aplicaciones:

a) Nada más inicializar la sesión del usuario podemos guardar ciertas variables de sesión con información del cliente:

```
$_SESSION['userAgent'] = $_SERVER['HTTP_USER_AGENT'];  
$_SESSION['IPaddress'] = $_SERVER['REMOTE_ADDR'];  
$_SESSION['LastActivity'] = $_SERVER['REQUEST_TIME'];
```

<https://www.php.net/reserved.variables.server>

USER_AGENT

En las cabeceras de cada mensaje entre cliente y servidor además de la cookie con el PHPSESSID también se especifica el user-agent, que incluye información sobre el cliente, la versión del navegador con el que está haciendo las peticiones...Podemos utilizar esta

información para que aunque se apropien del PHPSESSID, los atacantes necesiten las mismas condiciones que indica el user-agent.

Se guarda tras identificar al usuario y se hacen comprobaciones para cada script

```
$_SESSION['agent'] = sha1($_SERVER['HTTP_USER_AGENT']);
```

```
<?php
/*home.php*/
session_start();
if (isset($_SESSION['agent']))
{
if ($_SESSION['agent'] != sha1($_SERVER['HTTP_USER_AGENT']))
{
exit;
}
}
...
?>
```

IPADDRESS

Podemos usar la IP del cliente para confirmar que no hay modificaciones, por si alguien desde una ubicación distinta intenta hacerse pasar por él. Si se utiliza un proxy, REMOTE_ADDR es la dirección del mismo y no la del cliente, en ese caso se puede utilizar HTTP_X_FORWARDED_FOR o bien HTTP_CLIENTE_IP

```
if (!empty($_SERVER['HTTP_CLIENT_IP'])) {
    $ip = $_SERVER['HTTP_CLIENT_IP'];
} elseif (!empty($_SERVER['HTTP_X_FORWARDED_FOR'])) {
    $ip = $_SERVER['HTTP_X_FORWARDED_FOR'];
} else {
    $ip = $_SERVER['REMOTE_ADDR'];
}
```

LASTACTIVITY

Por temas de control de actividad se puede guardar el momento en que se realiza la petición al servidor web. La fecha de la última actividad no debe superar un umbral definido por nosotros. Si definimos un tiempo de expiración de sesión muy alto estamos dando más tiempo al usuario 'malicioso' a que pruebe y desarrolle sus ataques.

Tras hacer login y al crear la sesión nos guardaríamos el momento del último acceso:

```
$_SESSION['LAST_LOGIN'] = time(); // última identificación
```

```
$_SESSION['LastActivity'] = $_SERVER['REQUEST_TIME'];// última identificación
```

Y podríamos introducir la siguiente función para saber si tenemos que cancelar la sesión o no por haber pasado el tiempo límite.

```
function tiempo() {  
if (isset($_SESSION['LAST_LOGIN']) && (time() - $_SESSION['LAST_LOGIN'] >  
1800)) {  
if (isset($_COOKIE[session_name()])) {  
setcookie(session_name(), "", time() - 3600, "/");  
//limpiamos completamente el arraysuperglobal  
session_unset();  
//Eliminamos la sesión (archivo) del servidor  
session_destroy();  
}  
header(...) //redirigir al punto de partida para identificarse  
exit;  
}  
//...  
}
```

En caso de que suceda un cambio en alguno de los mencionados parámetros, o que la sesión del usuario haya caducado en el acceso al servidor web, debemos cerrar la sesión y salir.

b) Debemos definir una función que se encargue de destruir la sesión del usuario.

Para destruir una sesión en php debemos borrar todas las variables del array global `$_SESSION` mediante una llamada a `session_unset()`. La llamada a `session_destroy()` eliminará toda la información asociada con la sesión en el servidor.

```
functionlogOut() {  
    session_unset();
```

```
session_destroy();  
}
```

c) Regenerar ID de sesión

Una manera de proteger las aplicaciones frente a posibles robos del identificador de sesión, es mediante una simple regeneración de dicho id. De esta manera un atacante no podría utilizar el id apropiado ya que no sería el mismo que el generado por el servidor. Hay que pensar que el robo de id de sesión solo es útil para el atacante después de que el usuario se haya logueado y se haya creado la sesión. Por lo tanto es un buen momento para regenerar dicho id en tiempo de ejecución:

Por ejemplo:

```
<?php  
session_start();  
if (!isset($_SESSION['inicializada']))  
{  
    session_regenerate_id();  
    $_SESSION['inicializada'] = true;  
}  
?>
```

Esto nos protegería ante la fijación de la sesión ya que el id que pretendía usar el atacante, forzando al usuario a crear una sesión con él, habrá cambiado.

Aunque evidentemente, no solucionará nada si interceptan todas las comunicaciones, ya que todos los cambios de cookies de sesión viajan por las cabeceras de las peticiones y respuestas.

d) Cambia el nombre por defecto de la sesión PHPSESSID.

Este valor se establece con la función `session_name()`, con el valor de identificación propio como parámetro, antes de llamar a `session_start()`.

e) Si queremos asegurarnos de que nuestras páginas no sean cacheadas por ningún proxy intermedio, podemos incluir las siguientes directivas.

```
header("Cache-Control: no-store, no-cache, must-revalidate");  
header("Expires: Tue, 01 Jul 2001 06:00:00 GMT");
```

Adicionalmente, las opciones de configuración de

`session_cache_limiter()` y `session.cache_limiter` pueden ser usadas para generar automáticamente encabezados caché cuando se están usando sesiones.

Además de todo el desarrollador debe utilizar las configuraciones de forma óptima

<https://www.php.net/manual/es/session.security.php>

Algunas configuraciones que nos permite PHP

`_session.cookielifetime = 0`. El cero tiene un significado especial, le dice a los navegadores que no guarden una cookie permanentemente. Así pues, si se cierra el navegador, la session ID se borra inmediatamente. Se desaconseja variar este valor, y si se quiere una aplicación con autologin, existen alternativas mucho más seguras.

`_session.usecookies = On` y `_session.use_onlycookies = On`. Aunque las HTTP cookies tienen algunos problemas, es la forma más recomendable de manejar sesiones.

`_session.use_strictmode = On`. Esto previene al módulo de sesión iniciar una sesión ID sin inicializar. El módulo de sesión sólo acepta session ID válidas generadas por él mismo, rechazando cualquier session ID proporcionada por usuarios. Sesión ID injection puede hacerse con cookie injection a través de JavaScript. Se recomienda mantenerla en On.

`_session.cookiehttponly = On`. Rechaza el acceso a la cookie de sesión vía JavaScript. Esto previene el robo de cookies a través de JavaScript injection. Se puede usar session ID como CSRFprotectionkey, pero no es recomendable.

`_session.cookiesecure = On`. Permite el acceso a la cookie session ID sólo cuando el protocolo es HTTPS. Si tu aplicación es sólo HTTPS, es recomendable tener activada esta opción.

`_session.hashfunction = "sha256"`. Una función hash más fuerte generará una sessionID más fuerte. Cuanto más complejo sea el cifrado mejor: sha384, sha512... Este valor se configura en el php.ini

`_session.hash_bits_percharacter = 5`. Es una traba más para cuando el atacante trate de adivinar el session ID. El ID será más corto pero usará más caracteres.

